

Fiche récapitulative – STA211

Éléments fondamentaux pour méthodes supervisées

Apprentissage statistique, compromis biais-variance, choix de modèles et validation

Vincent Audigier, Ndèye Niang (CNAM)

14 juin 2026

Résumé

Cette fiche présente les fondements des méthodes supervisées en data-mining. Après avoir posé le cadre de l'apprentissage statistique (régression, classification, fonctions de perte), elle développe le compromis biais-variance, puis les stratégies de choix de modèles : critères pénalisés (AIC, BIC), méthodes pas-à-pas, découpage apprentissage/test, validation croisée, et courbe ROC/AUC. Des exemples en R/Python illustrent chaque concept.

Introduction

Le **data-mining** vise à identifier des structures au sein de données. On distingue :

- Les **modèles** : expliquent et/ou prédisent une variable réponse Y à partir de variables explicatives X .
- Les **patterns** : régularités locales sans variable cible.

Régression *vs* classification

- **Régression** : Y est continue.
- **Classification supervisée (discrimination)** : Y est qualitative.

Méthodes paramétriques *vs* non-paramétriques

- **Paramétriques** : forme a priori pour la fonction de lien f , modulée par des paramètres (ex. régression linéaire, logistique).
- **Non-paramétriques** : aucune forme spécifique supposée (ex. arbres, forêts aléatoires, SVM, réseaux de neurones).

TABLE 1 – Typologie des méthodes supervisées

Régression		Classification
Paramétrique	Rég. linéaire	Rég. logistique, ALD, modèles add.
Non-paramétrique	Arbres, forêts aléatoires, réseaux de neurones, splines	Arbres, forêts aléatoires, SVM, réseaux de neurones

Apprentissage statistique

Notations et contexte

Soit Y la variable réponse et $\mathbf{X} = (X_1, \dots, X_p)$ le vecteur des p variables explicatives observé sur n individus :

$$\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\} \in \mathcal{X} \times \mathcal{Y}$$

Deux objectifs :

- **Prédiction** : pour $\mathbf{x}_0$, prédire $\hat{y}_0$ le plus proche possible de y_0 .
- **Estimation** : approcher la fonction f qui relie Y à X .

Fonctions de perte

Régression : On suppose $Y = f(X) + \varepsilon$ avec $\mathbb{E}[\varepsilon] = 0$ et $\text{Var}[\varepsilon] = \sigma^2$.

Fonction de contraste des **moindres carrés** :

$$\gamma(\hat{f}, (x, y)) = (\hat{f}(x) - y)^2$$

Perte (espérance) :

$$P_\gamma(\hat{f}) = \mathbb{E}_{X,Y}[(Y - \hat{f}(X))^2]$$

Classification : On estime la probabilité *a posteriori* $\mathbb{P}(Y = m_q|X)$.

Contraste **0-1** :

$$\gamma(\hat{f}, (x, y)) = 1_{\hat{f}(x) \neq y}$$

Perte (taux d'erreur) :

$$P_\gamma(\hat{f}) = \mathbb{P}(\hat{f}(X) \neq Y)$$

En estimation, on minimise :

$$\mathbb{E} \left[\sum_{q=1}^k |\hat{p}(Y = m_q|X) - \mathbb{P}(Y = m_q|X)| \right]$$

Sur-ajustement (overfitting)

Un modèle trop complexe (ex. $k = 1$ en k-NN) donne une erreur d'apprentissage nulle mais ne généralise pas. Il faut un échantillon **indépendant** pour estimer la perte.

Données déséquilibrées

Cas fréquent : une modalité minoritaire (ex. 30% d'impayés). Le contraste 0-1 étant symétrique, le modèle ignore la classe rare.

Solutions :

- **Contraste asymétrique** : $\gamma = w_0 1_{\hat{f}(x) \neq 0, y=0} + w_1 1_{\hat{f}(x) \neq 1, y=1}$
- **Sur-échantillonnage** : tirage avec remise dans la classe minoritaire.
- **Sous-échantillonnage** : tirage dans la classe majoritaire.
- **SMOTE** : génère des individus fictifs sur des segments entre voisins minoritaires (Chawla et al., 2002).

Algorithme SMOTE (sur-échantillonnage) : Pour chaque individu i de la classe minoritaire :

1. Identifier ses $k = 5$ plus proches voisins (minoritaires).
2. Tirer un individu i' au hasard parmi les k .
3. Pour $j = 1, \dots, p$: $x_{i''j} \leftarrow x_{ij} + u \times (x_{ij} - x_{i'j})$, avec $u \sim \mathcal{U}[0, 1]$.

Compromis biais - variance

La qualité d'un modèle est liée à sa complexité. Un modèle trop simple (biais élevé) sous-ajuste, un modèle trop complexe (variance élevée) sur-ajuste.

Décomposition de l'erreur

En régression (MSE), l'erreur espérée se décompose :

$$\mathbb{E}[(Y - \hat{f}(X))^2] = \underbrace{\text{Var}[\hat{f}(X)]}_{\text{variance}} + \underbrace{(\mathbb{E}[\hat{f}(X)] - f(X))^2}_{\text{biais}^2} + \underbrace{\sigma^2}_{\text{bruit}}$$

Illustration (k-NN sur German Credit)

Avec k petit (modèle complexe) : erreur train faible, erreur test élevée (surapprentissage). Avec k optimal ($k \approx 10$) : meilleur compromis. Au-delà : erreur test augmente (biais trop fort).

Taux de mauvais classement ↓		
k	Erreur train	Erreur test
1	0%	~45%
10	~25%	~30%
50	~35%	~35%

FIGURE 1 – Compromis biais-variance pour k-NN sur German Credit

Choix de modèles

Critères pénalisés

On ajoute une pénalité de complexité au risque empirique :

$$\mathcal{L}(\hat{f}) = \frac{1}{n} \sum_{i=1}^n \gamma(\hat{f}, (\mathbf{x}_i, y_i)) + \text{pénalité}$$

AIC (Akaike Information Criterion) :

$$\text{AIC} = -2 \ln L(\hat{\theta}) + 2k$$

où k est le nombre de paramètres. À privilégier pour la **prédiction**.

BIC (Bayesian Information Criterion) :

$$\text{BIC} = -2 \ln L(\hat{\theta}) + \ln(n) k$$

Pénalité plus forte que l'AIC ($\ln(n) > 2$ pour $n \geq 8$). À privilégier pour l'**estimation** (sélection du vrai modèle asymptotiquement).

Méthodes pas-à-pas :

- **Descendante** : part du modèle complet, élimine les variables une à une.
- **Ascendante** : part du modèle vide, ajoute les variables une à une.
- **Progressive** : ascendante avec possibilité d'élimination.

Limites des critères pénalisés :

- Nécessitent un modèle paramétrique (vraisemblance).
- Ne comparent pas des modèles de familles différentes.
- Le nombre de paramètres ne reflète pas toujours la complexité (ex. régression ridge).

Approches empiriques

Découpage test/apprentissage :

1. Séparer les données en $\mathcal{A}$ (apprentissage) et $\mathcal{T}$ (test).
2. Ajuster chaque modèle sur $\mathcal{A}$.
3. Choisir celui qui minimise l'erreur sur $\mathcal{T}$.
4. Ré-ajuster le modèle retenu sur $\mathcal{A} \cup \mathcal{T}$.

Validation croisée K -fold :

1. Découper l'échantillon en K blocs de même taille.
2. Pour chaque bloc : les $K - 1$ autres servent d'apprentissage, le bloc sert de test.
3. Moyenner les K erreurs de test.

$K = 10$ est le choix usuel. $K = n = \text{leave-one-out}$ (LOO).

Mesure objective de la perte : 3 échantillons : **apprentissage** (ajuster), **validation** (choisir), **test** (mesure finale). Proportion typique : 1/2, 1/4, 1/4.

Complément pour la classification : ROC et AUC

Pour une variable binaire, on peut faire varier le seuil s de décision $\hat{p}(Y = 1|X) > s$.

Courbe ROC : taux de vrais positifs en fonction du taux de faux positifs pour tous les seuils $s \in [0, 1]$.

AUC (Area Under the Curve) :

- AUC = 0.5 : modèle aléatoire (aucun pouvoir prédictif).
- AUC = 1 : modèle parfait.
- En pratique : AUC > 0.7 acceptable, > 0.8 excellent.

Code Python**Validation croisée avec scikit-learn**

```

1 from sklearn.model_selection import cross_val_score, train_test_split
2 from sklearn.neighbors import KNeighborsClassifier
3 from sklearn.datasets import make_classification
4 import numpy as np
5
6 X, y = make_classification(n_samples=200, n_features=2,
7                           n_redundant=0, random_state=42)
8 X_train, X_test, y_train, y_test = train_test_split(
9     X, y, test_size=0.33, random_state=42)
10
11 for k in [1, 5, 10, 15, 30]:
12     model = KNeighborsClassifier(n_neighbors=k)
13     scores = cross_val_score(model, X_train, y_train, cv=10)
14     model.fit(X_train, y_train)
15     test_score = model.score(X_test, y_test)
16     print(f"k={k:2d} | CV: {scores.mean():.3f} (+/-{scores.std():.2f}) "
17           f"| Test: {test_score:.3f}")

```

SMOTE avec imbalanced-learn

```

1 from imblearn.over_sampling import SMOTE
2 from collections import Counter
3
4 smote = SMOTE(random_state=42)
5 X_res, y_res = smote.fit_resample(X_train, y_train)
6 print("Avant SMOTE:", Counter(y_train))
7 print("Après SMOTE:", Counter(y_res))

```

Courbe ROC et AUC

```

1 from sklearn.metrics import roc_curve, auc
2 import matplotlib.pyplot as plt
3
4 model = KNeighborsClassifier(n_neighbors=10)
5 model.fit(X_train, y_train)
6 y_prob = model.predict_proba(X_test)[: , 1]
7 fpr, tpr, _ = roc_curve(y_test, y_prob)
8 roc_auc = auc(fpr, tpr)
9
10 plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.3f}")
11 plt.plot([0,1], [0,1], 'k--')
12 plt.xlabel("Taux faux positifs")
13 plt.ylabel("Taux vrais positifs")
14 plt.legend()
15 plt.show()

```

Code R

Validation croisée avec caret

```

1 library(caret)
2 library(class)
3
4 data(GermanCredit)
5 train_idx <- createDataPartition(GermanCredit$Class,
6                                   p = 0.67, list = FALSE)
7 train <- GermanCredit[train_idx, ]
8 test  <- GermanCredit[-train_idx, ]
9
10 ctrl <- trainControl(method = "cv", number = 10)
11 for (k in c(1, 5, 10, 15, 30)) {
12   model <- train(Class ~ ., data = train, method = "knn",
13                 trControl = ctrl, tuneGrid = data.frame(k = k))
14   cat(sprintf("k=%2d | CV: %.3f | Test: %.3f\n",
15               k, max(model$results$Accuracy),
16                   predict(model, test) |> (\(x)
17                     mean(x == test$Class))()))
18 }

```

SMOTE avec le package UBL

```

1 library(UBL)
2 smote_train <- SmoteClassif(Class ~ ., train, C.perc = "balance")
3 table(smote_train$Class)

```

Courbe ROC

```

1 library(pROC)
2 model <- knn3(Class ~ ., train, k = 10)
3 prob <- predict(model, test)[, 2]
4 roc_obj <- roc(test$Class, prob)
5 plot(roc_obj, print.auc = TRUE)

```

Questions clés (QCM)

1. Quelle est la fonction de contraste usuelle en régression ?

- (a) Contraste 0-1
- (b) Contraste des moindres carrés
- (c) Contraste log-vraisemblance
- (d) Contraste asymétrique

Réponse : b (moindres carrés).

2. En classification, la perte $P_\gamma(\hat{f}) = \mathbb{P}(\hat{f}(X) \neq Y)$ correspond à :

- (a) L'AUC
- (b) Le taux de mauvais classement
- (c) La somme des carrés résiduels
- (d) Le BIC

Réponse : b (taux d'erreur).

3. Que se passe-t-il quand $k = 1$ en k-NN sur l'erreur d'apprentissage ?

- (a) Erreur maximale
- (b) Erreur nulle (sur-apprentissage)
- (c) Erreur égale à l'erreur de test
- (d) Le modèle ne converge pas

Réponse : b (erreur nulle, chaque point est son propre plus proche voisin).

4. La procédure SMOTE permet de :

- (a) Réduire la dimension
- (b) Générer des individus fictifs pour la classe minoritaire
- (c) Sélectionner des variables
- (d) Calculer l'AUC

Réponse : b (sur-échantillonnage synthétique).

5. Dans la décomposition biais-variance, un modèle trop simple a :

- (a) Un biais élevé, une variance faible
- (b) Un biais faible, une variance élevée
- (c) Un biais élevé, une variance élevée
- (d) Un biais nul, une variance nulle

Réponse : a (sous-apprentissage).

6. Le critère AIC pénalise la log-vraisemblance par :

- (a) $2k$
- (b) $\ln(n) k$
- (c) k
- (d) $\sqrt{k}$

Réponse : a ($\text{AIC} = -2 \ln L + 2k$).

7. Pour $n = 100$, quel critère pénalise le plus la complexité ?

- (a) AIC
- (b) BIC
- (c) Les deux pénalisent autant
- (d) Aucun des deux

Réponse : b ($\text{BIC} : \ln(100) \approx 4.6 > 2$).

8. En validation croisée K -fold, que vaut K pour le leave-one-out ?

- (a) $K = 10$
- (b) $K = 5$
- (c) $K = n$
- (d) $K = 1$

Réponse : c ($K = n$, chaque individu est son propre bloc de test).

9. Une AUC de 0.5 signifie que le modèle :

- (a) Est parfait
- (b) Est aléatoire (aucun pouvoir prédictif)
- (c) Est excellent
- (d) A un biais nul

Réponse : b (AUC=0.5 = prédiction aléatoire).

10. Quel est l'ordre des 3 échantillons pour une mesure objective de la perte ?

- (a) Test $\rightarrow$ Validation $\rightarrow$ Apprentissage
- (b) Apprentissage $\rightarrow$ Validation $\rightarrow$ Test
- (c) Apprentissage $\rightarrow$ Test $\rightarrow$ Validation
- (d) Validation $\rightarrow$ Apprentissage $\rightarrow$ Test

Réponse : b (apprentissage : ajuster, validation : choisir, test : mesurer).

Références

- [1] N. V. Chawla, K. W. Bowyer, L. O. Hall, W. P. Kegelmeyer. *SMOTE: Synthetic Minority Over-sampling Technique*. J. Artif. Intell. Res., 16:321–357, 2002.
- [2] T. Hastie, R. Tibshirani, J. Friedman. *The Elements of Statistical Learning*. Springer, 2009.
- [3] G. Saporta. *Probabilités, analyse des données et statistique*. Editions Technip, 2006.
- [4] P.-A. Cornillon, E. Matzner-Lober. *Régression avec R*. Springer, 2010.
- [5] S. Tufféry. *Data Mining et Statistique décisionnelle*. Editions Technip, 2007.